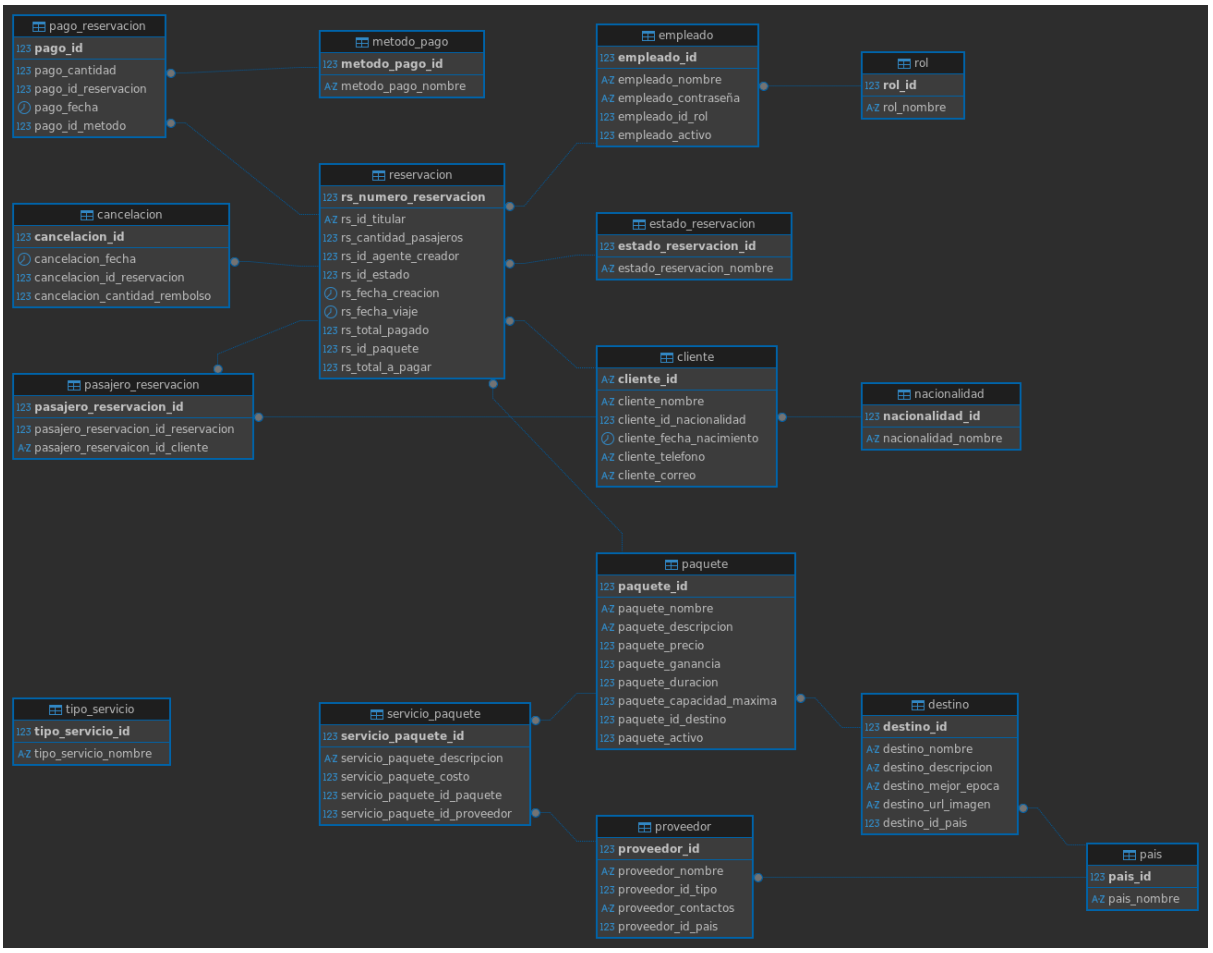
MANUAL TÉCNICO Horizontes sin límites

La práctica fué realizada usando java 21, para el backend, se utilizó angular par el frontend y sql como base de datos
Se usó NetBeans como editor de código, Linux Pop Os 22 como sistema operativo

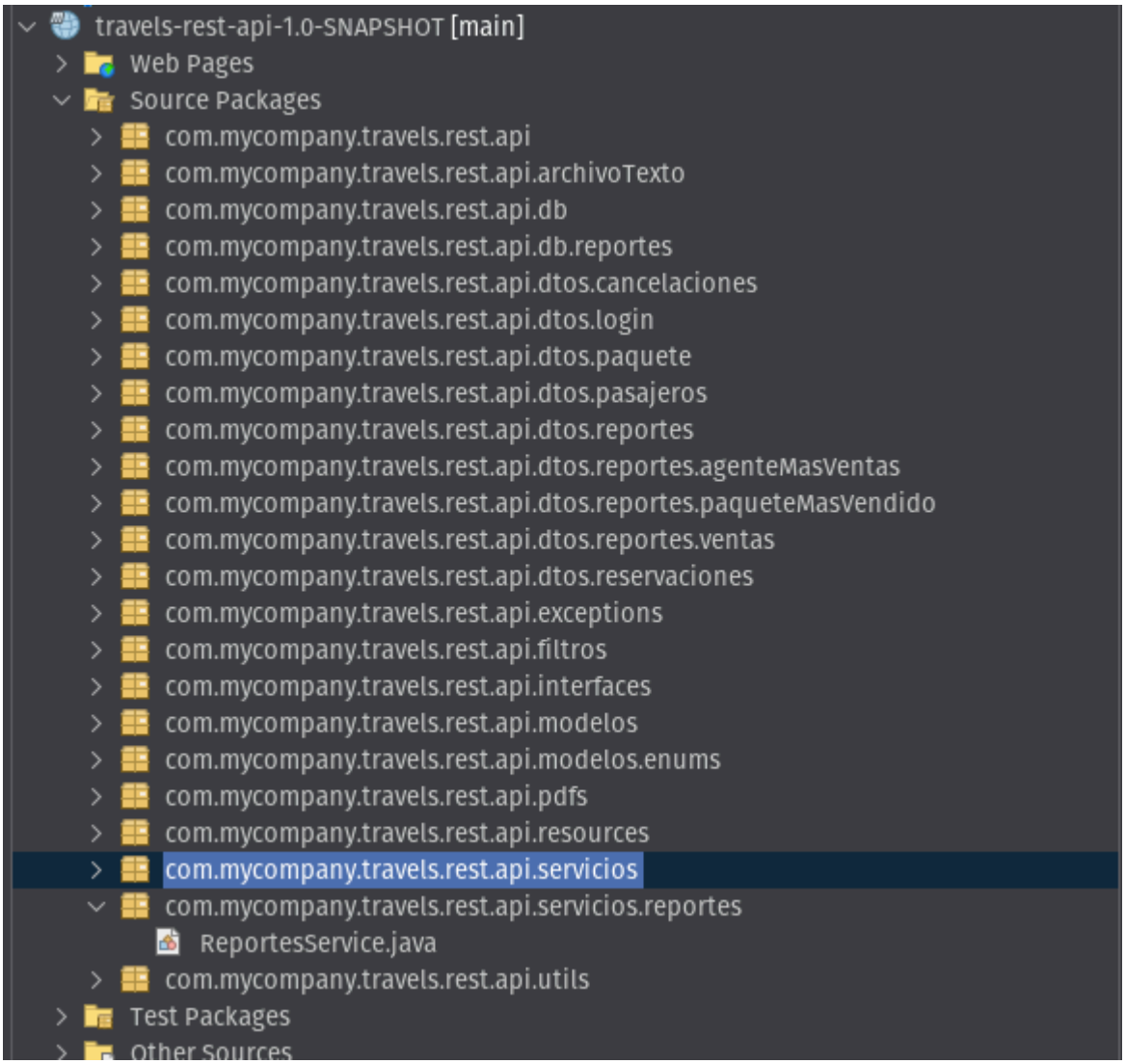
Base de datos

se utilizaron 16 entidades para realizar este proyecto las cuales son:



Como detalle, se implementó el ON DELETE CASCADE si se elimina algun registro, se elimina todos los relacionados

Ordenamiento de paquetes código fuente



El el backend se utilizó:
Reousrces,
CrudService,
Db,
DTO
Modelo

Crud service:

es una clase que recibe la entidad y realiza las validaciones necesarias antes de hacer registros en la db:

```java
/**
 *
 * @author edu
 */
public class CancelacionesCrudService {
    private final CancelacionesDB db = new CancelacionesDB();

    public void cancelarReservacion(CancelacionRequest cancelacion) throws ExceptionGenerica{

        LocalDate fechaViaje = db.buscarFechaViaje(cancelacion.getIdReservacion());

        verificarQuenoEsteCancelada(cancelacion.getIdReservacion());
        verificarPlazoLimite(fechaViaje);

        long diasRestantes = obtenerDiasRestantes(fechaViaje);
        double totalPagado = db.obtenerTotalPagado(cancelacion.getIdReservacion());
        double reembolso = calcularReemboso(totalPagado, diasRestantes);

        Cancelacion canc = new Cancelacion(cancelacion.getFechaCancelacion(),
        reembolso, cancelacion.getIdReservacion());
        db.registrarCancelacion(canc);
    }



    private void verificarQuenoEsteCancelada(int idReservacion) throws ExceptionGenerica{
        int estadoReservacion = db.buscarEstadoReservacion(idReservacion);
        if(estadoReservacion ==3){ // si no se encuentra en el rango
            throw new ExceptionGenerica("la reservación ya está cancelada");
        }
    }
```

en este ejemplo se realiza la cancelación de una reservación, no solo se escribe en la db

Clase db:

esta es una clase que solo recibe la entidad, y realiza registros en la base de datos: no realiza validaciones a diferencia de una clase Crud service:

```java
*/
public class CancelacionesDB {

    private static final String CANCELACION = "UPDATE reservacion  SET rs_id_estado = 3 where rs_numero_reservacion = ?";

    private static final String CREAR_CANCELACION = "INSERT INTO cancelacion"
            + " (cancelacion_fecha, cancelacion_id_reservacion, cancelacion_cantidad_rembolso)"
            + "VALUES (?, ?, ?)";

    private static final String BUSCAR_ESTADO_RESERVACION = "select rs_id_estado from reservacion where rs_numero_reservacion = ?";

    private static final String BUSCAR_FECHA_VIAJE = "SELECT rs_fecha_viaje from reservacion where rs_numero_reservacion = ?";


    private static final String BUSCAR_MONTO_PAGADO = "select rs_total_pagado from reservacion where rs_numero_reservacion = ?";

    public void registrarCancelacion(Cancelacion cancelacion) throws ExceptionGenerica {
        try (Connection conexion = ConexionDB.getConnection();
                PreparedStatement ps = conexion.prepareStatement(CREAR_CANCELACION)) {

            ps.setDate(1, Date.valueOf(cancelacion.getFechaCancelacion()));
            ps.setInt(2, cancelacion.getIdReservacion());
            ps.setDouble(3, cancelacion.getCantidadReembolsada());
            this.marcarComoCancelada(cancelacion.getIdReservacion());
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new ExceptionGenerica("Error al registrar cancelación: " + e.getMessage());
        }
    }
```

Clase DTO:

esta clase se para llevar datos del backend al frontend, a diferencia de la clase modelo, representan una clase mas completa y no con información mínima como son los modelos, en esta ocasión se usaron algunos modelos como dtos

```java
public class PaqueteGeneral {

    private Paquete paquete;
    private ArrayList<Paquete_servicio> servicios;
    private ArrayList<Paquete_servicio> nuevosServicios;
```

Clase MODELO:

representa la información mínima para llevar un registro en la base de datos

```java
/**
 *
 * @author edu
 */
public class Cancelacion {
    private LocalDate fechaCancelacion;
    private double cantidadReembolsada;
    private int idReservacion;

    public Cancelacion(LocalDate fechaCancelacion, double cantidadReembolsada, int idReservacion) {
        this.fechaCancelacion = fechaCancelacion;
        this.cantidadReembolsada = cantidadReembolsada;
        this.idReservacion = idReservacion;
    }

    public LocalDate getFechaCancelacion() {
        return fechaCancelacion;
    }

    public double getCantidadReembolsada() {
        return cantidadReembolsada;
    }

    public int getIdReservacion() {
        return idReservacion;
    }


}
```

Clase RESOURCE:

representa un endpoint de la api, tiene metodos como get, put, delete, post

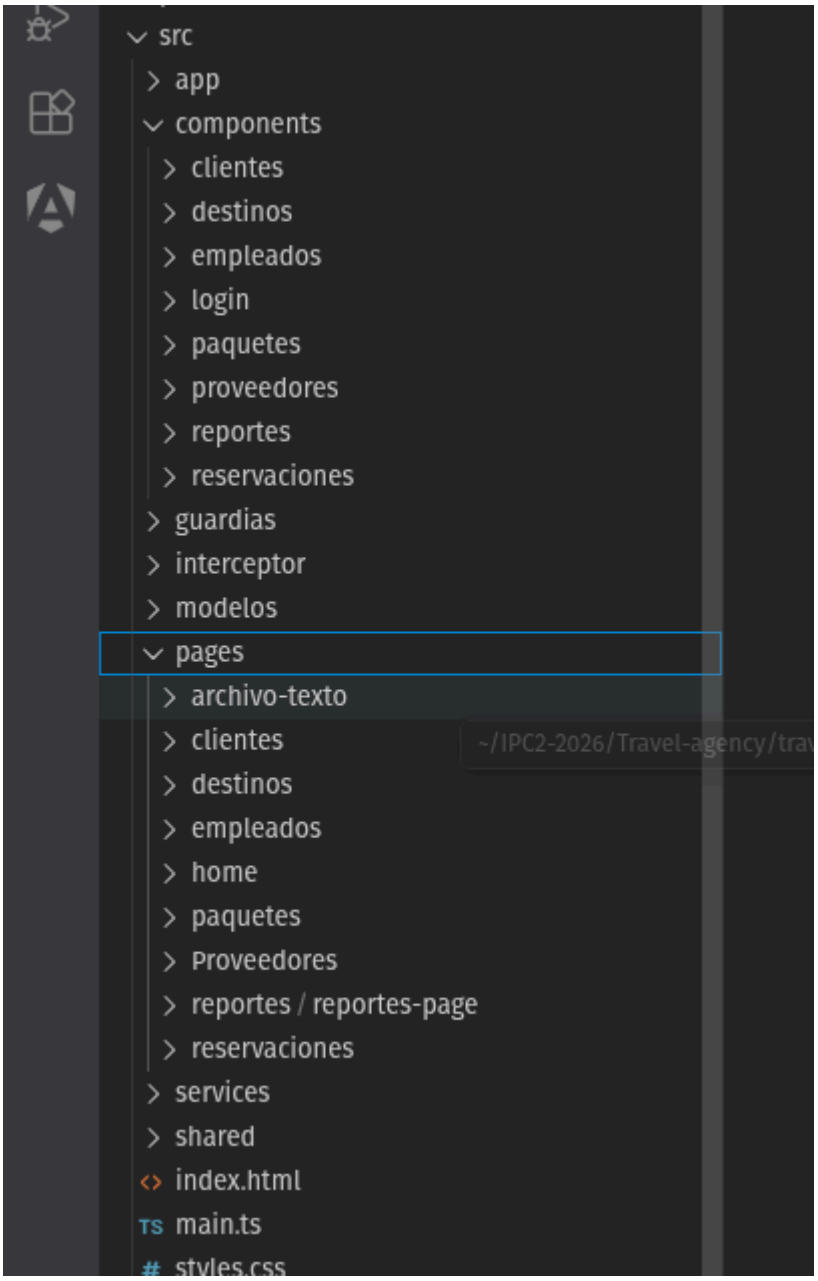
```java
import jakarta.servlet.http.HttpServletResponse;

/**
 *
 * @author edu
 */
@WebServlet(name = "DestinosResource", urlPatterns = {"/api/destinos/*"})
public class DestinosResource extends HttpServlet {

    private final Gson gson = new Gson();
    private DestinosCrudService crudService = new DestinosCrudService();
    private EscritorJson escritor = new EscritorJson();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
```

Ordenamiento de paquetes FRONTEND



COMPONENTE:

los componentes se dividen en tarjetas y forms
las tarjetas muestran información importante, mientras los forms permiten la creación o edición de una entidad

PAGES:
una pagina contiene varios componentes y es el contenedor
existen varios tipos de pages

edit-page = pagina que busca una entidad por id y la pone en un componente si la encuentra para editar

form-page = es una pagina que el componente formulario de una entidad

page = una pagina normal que permite realizar varias acciones, ejemplo: pagina reservaciones, permite ver reservaciones, crear nueva reservación.